

Versi 2.1
Agustus 2026



PEMROGRAMAN MOBILE

MODUL 2 - STATE, HOOKS, & NAVIGASI PAGE

Penulis:

Eko Felix Nur Noftianto, M.Kom.

Wira Yudha Aji Pratama

Fatahillah Al-Fatih

LABORATORIUM INFORMATIKA
Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	1
PENDAHULUAN.....	4
1. TUJUAN.....	4
2. TARGET MODUL.....	4
4. PERSIAPAN.....	4
4. KATA KUNCI.....	4
MATERI.....	5
1. STATE.....	5
2. HOOKS.....	6
2.1. USESTATE.....	7
2.2. USEEFFECT.....	9
2.3. USEREF.....	10
2.4. USECONTEXT.....	12
CODELAB-1 (5-10 menit, kerjakan di lab).....	17
2.5. USEMEMO.....	18
2.6. USELOCALPARAMS().....	21
2. PENAMAAN FILE UNTUK ROUTING.....	23
2.1. NORMAL PAGE.....	23
2.2. INDEX PAGE.....	24
2.3. PRIVATE ROUTE.....	24
2.4. ROUTE GROUP.....	24
2.5. DYNAMIC ROUTE.....	25
2.6. CATCH ALL ROUTE.....	25
2.7. LAYOUT PAGE.....	26
3. NAVIGASI ANTAR PAGE.....	28
3.1. NAVIGASI MENGGUNAKAN LINK.....	28
3.2. NAVIGASI MENGGUNAKAN ROUTER.....	29
A. PUSH() & BACK().....	32
B. REPLACE().....	33
C. CANGOBACK().....	34
D. DISMISS().....	34
E. DISMISSALL().....	37
G. SETPARAMS().....	38
4. PRAKTIK.....	39
CODELAB-2 (10 menit, kerjakan di lab).....	40
TUGAS PEKAN DEMO.....	42
1. DESKRIPSI.....	42





2. KRITERIA PENILAIAN..... 43

3. SKALA PENILAIAN.....43

KESIMPULAN AKHIR MODUL..... 44



PENDAHULUAN

1. TUJUAN

1. Memungkinkan mahasiswa memahami konsep State dan React Hooks dalam React Native.
2. Memberikan pemahaman yang jelas kepada mahasiswa tentang arsitektur routing berbasis file menggunakan Expo Router.
3. Membekali mahasiswa dengan pengetahuan yang diperlukan untuk membuat layar yang dinamis dan mengelola navigasi halaman yang lancar dalam aplikasi mobile.

2. TARGET MODUL

1. Mahasiswa mampu mengelola state aplikasi menggunakan React Hooks utama.
2. Mahasiswa mampu merancang dan menyusun nested routes, dynamic routes, dan catch-all routes menggunakan Expo Router.
3. Mahasiswa mampu melakukan navigasi layar secara programmatic maupun declarative sambil meneruskan parameter dinamis.

4. PERSIAPAN

1. Mahasiswa Muslim dapat membaca doa berikut sebelum memulai pembelajaran:

رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا

2. Laptop atau komputer pribadi (Windows/Mac).
3. Visual Studio Code.
4. Koneksi internet.
5. Telepon seluler (Android / iOS).
6. Motivasi dan komitmen yang kuat untuk belajar.

5. KATA KUNCI

State, React Hooks, Expo Router, File-based Routing, Dynamic Routes, Navigation, Stack Layout.



MATERI

Baik, sekarang kita akan melanjutkan pembelajaran dari **Modul 1**. Materi yang dibahas sebelumnya cukup banyak, bukan? 😊

Namun, percaya atau tidak, materi tersebut masih belum mencakup semuanya. Sebenarnya, masih banyak topik lain yang dilewatkan dalam Modul 1 karena terlalu banyak hal yang harus dibahas sekaligus.

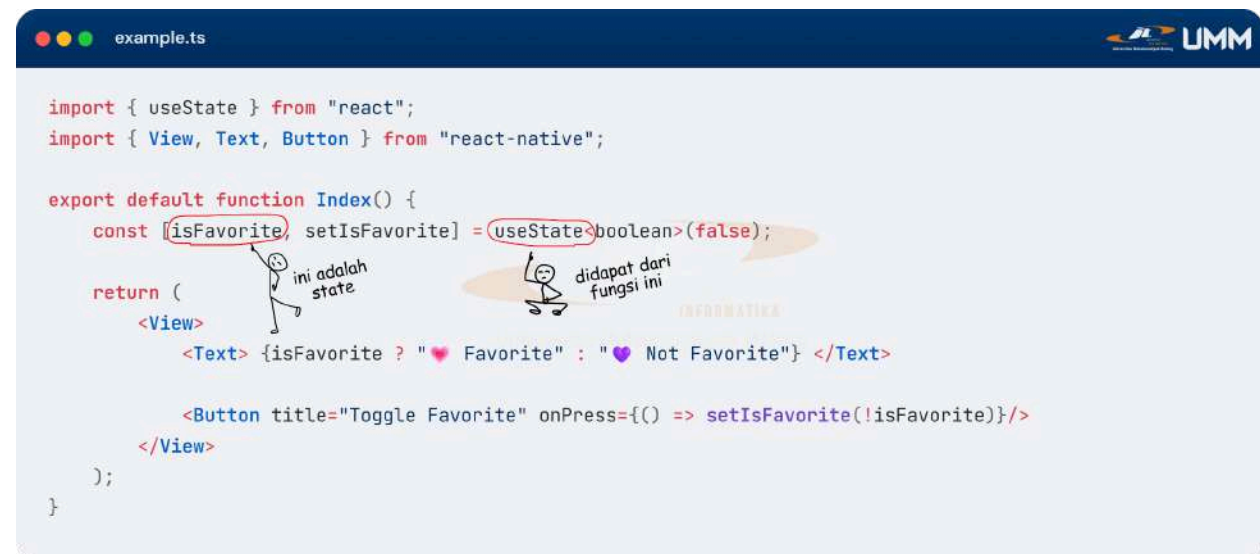
Jadi, dalam modul ini, kita akan membahas kembali beberapa topik penting yang sebelumnya belum dibahas. Pada saat yang sama, kita juga akan mulai mempelajari topik utama **Modul 2: State, Hooks, & Page Navigation**.

Di akhir modul ini, kamu akan memahami cara *berpindah antar-screen*, mengatur aplikasi menjadi *beberapa halaman*, dan menciptakan pengalaman pengguna yang lebih baik melalui navigasi yang tepat. Secara bertahap, kita akan membangun pemahaman berdasarkan dasar-dasar yang telah kamu pelajari pada Modul 1.

1. STATE

State adalah variabel yang dapat berubah seiring waktu selama aplikasi berjalan. Namun, berbeda dengan variabel biasa, **ketika sebuah state berubah, React Native akan secara otomatis me-render ulang (re-render) layar**, sehingga pengguna dapat langsung melihat nilai yang telah diperbarui.

Contoh kode:



```
example.ts
import { useState } from "react";
import { View, Text, Button } from "react-native";

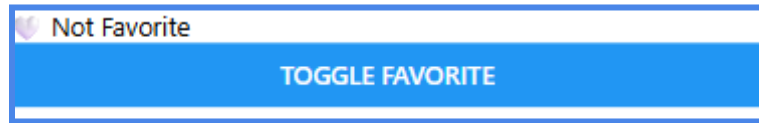
export default function Index() {
  const [isFavorite, setIsFavorite] = useState<boolean>(false);

  return (
    <View>
      <Text> {isFavorite ? "♥ Favorite" : "♥ Not Favorite"} </Text>

      <Button title="Toggle Favorite" onPress={() => setIsFavorite(!isFavorite)} />
    </View>
  );
}
```



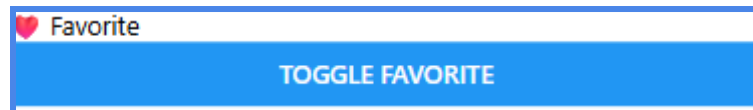
Output:



Sebagai contoh, bayangkan kamu memiliki tombol **"Toggle Favorite"**.

Ketika nilai *isFavorite* adalah **false**, warna ikon adalah **putih**. Kemudian, ketika kamu menekan tombol tersebut, nilainya berubah menjadi **true**.

Dan coba tebak apa yang terjadi? React Native secara otomatis **me-render ulang** komponen tersebut, sehingga warna ikon berubah menjadi merah. Jika kamu menekannya lagi, nilainya kembali menjadi **false** dan ikon kembali berwarna **putih**.



Variabel biasa tidak bekerja seperti itu. Meskipun nilainya berubah, UI tidak akan diperbarui secara otomatis.

Itulah alasan kita menggunakan state. State memungkinkan UI *bereaksi* terhadap perubahan nilainya secara real-time. Setiap kali state berubah, React Native secara otomatis memperbarui layar agar mencerminkan nilai terbaru.

2. HOOKS

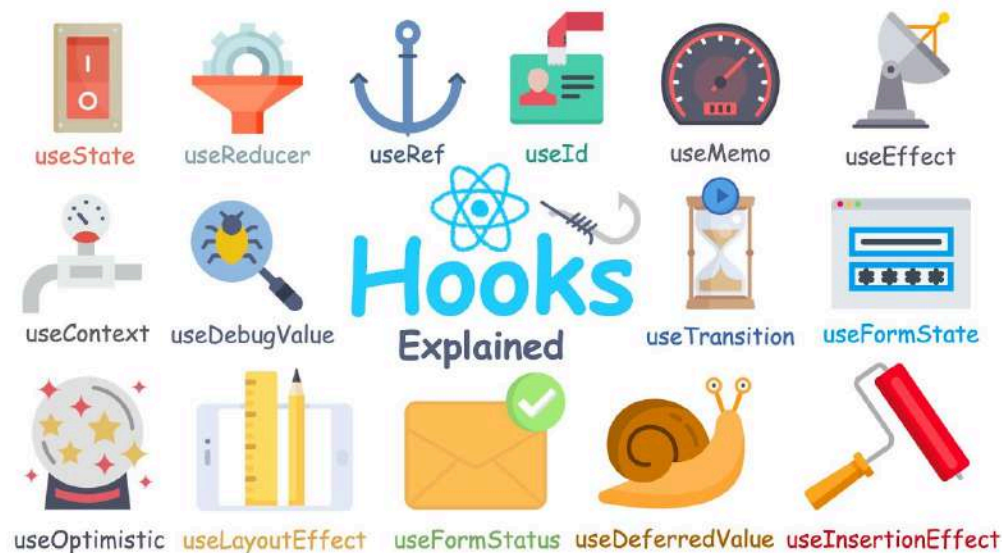


Image 2.1



Oke, penjelasan di atas menjelaskan apa itu *state*. Namun, **state tidak dapat digunakan begitu saja tanpa hook**, terutama hook `useState()` yang kita gunakan sebelumnya. Karena *state* merupakan salah satu **mekanisme yang disediakan oleh React**, kita menggunakan Hooks untuk menghubungkan komponen kita dengan mekanisme tersebut.

Jadi, apa itu Hooks? Hooks adalah *fungsi khusus yang disediakan oleh React* yang memungkinkan **function component terhubung dengan mekanisme bawaan React**, seperti *state, effects, refs, dan context*. Hooks biasanya memiliki *nama yang diawali dengan use*, seperti `useState`, `useRef`, `useEffect`, dan sebagainya.

2.1. USESTATE

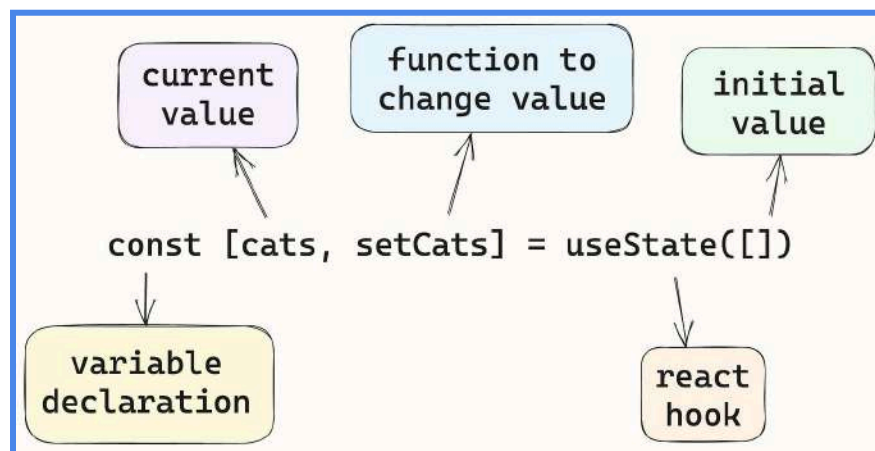


Image 2.1.1

Pola penulisan: `const [isFavorite, setIsFavorite] = useState<boolean>(false);`

Mari kita uraikan. Pertama, kita tidak perlu membahas *Hooks* atau *React functions* terlebih dahulu. Kita akan fokus pada **sintaks TypeScript**.

Apa arti pola penulisan: `[variable1, variable2] = variable3`?

Artinya, **variable1** dan **variable2** masing-masing menerima *nilai* dari **variable3**. Bayangkan **variable1** dan **variable2** sebagai anak-anak dari **variable3** yang meminta *permen*. Tentu saja, *variable3* harus memiliki permen terlebih dahulu, bukan? Jika tidak ada permen, apa yang bisa diberikan kepada mereka?

Sebagai contoh, jika *variable3* memiliki permen seperti:

```
const variable3 = ["Kopiko", "Milkita"];
```

Maka masing-masing anak akan menerima:

```
variable1 = "Kopiko"
```

dan

```
variable2 = "Milkita".
```

Hal tersebut bergantung pada posisinya (*indeks 0 atau 1*).



Jadi, pada contoh kita sebelumnya:

```
example.ts

export default function Index() {
  const [isFavorite, setIsFavorite] = useState<boolean>(false);

  return (
    <View>
      <Text> {isFavorite ? "♥ Favorite" : "♥ Not Favorite"} </Text>
    </View>
  );
}
```

isFavorite adalah variabel yang *menerima* atribut dari **useState**, begitu juga dengan **setIsFavorite**.

Pertanyaannya, apa yang **isFavorite** terima dan apa yang **setIsFavorite** terima dari fungsi **useState()**?

Variabel **isFavorite** akan menjadi **state**. Sementara itu, **setIsFavorite** dapat mengubah nilai **state** dan *me-render* ulang komponen. Kamu harus menggunakan fungsi ini untuk mengubah **state**. Jika tidak, **React tidak akan mengetahui bahwa state telah berubah**.

Kamu dapat melihat contoh cara menggunakannya pada contoh berikut:

```
example.ts

<View>
  <Text> {isFavorite ? "♥ Favorite" : "♥ Not Favorite"} </Text>

  <Button title="Toggle Favorite" onPress={() => setIsFavorite(!isFavorite)} />
</View>
```

Artinya, “ketika tombol diklik, nilai **isFavorite** berubah menjadi nilai kebalikannya (jika **true**, maka menjadi **false**; jika **false**, maka menjadi **true**).”

Selanjutnya, mari kita lihat pola berikutnya: `useState<boolean>(false)`

Di sini, “**useState**” adalah **Hook Function**.

<boolean> adalah *generic type argument*. Sebenarnya, bagian ini *opsional*, tetapi disarankan karena secara eksplisit memberi tahu TypeScript **type data** yang harus dimiliki oleh **state**. Jika tidak diberikan, tipenya biasanya akan ditentukan secara otomatis berdasarkan **nilai awal (initial value)**. Jika **tidak ada nilai awal**, tipenya akan menjadi **undefined**.

Terakhir, **(false)** adalah *nilai awal (initial value)* dari **state** saat komponen pertama kali di-render. Jika dibiarkan *kosong*, nilainya akan menjadi **undefined**.



TIPS:

```
const [location, setLocation] = useState<Location.LocationObject | null>(null);
```

Di VS Code, kamu dapat mengenali pola kode tertentu berdasarkan warnanya. Misalnya, warna kuning biasanya menunjukkan fungsi, putih menunjukkan variabel biasa, biru-hijau menunjukkan object, biru atau merah muda menunjukkan *keyword*, hijau menunjukkan komentar, dan sebagainya. Setidaknya, seperti itulah tampilannya di VS Code saya. Hal ini bergantung pada *theme* yang kamu gunakan. Kamu dapat melihat *theme* yang kamu gunakan dan mempelajari pola warnanya sendiri, sehingga kode akan menjadi jauh lebih mudah untuk dibaca.

2.2. USEEFFECT

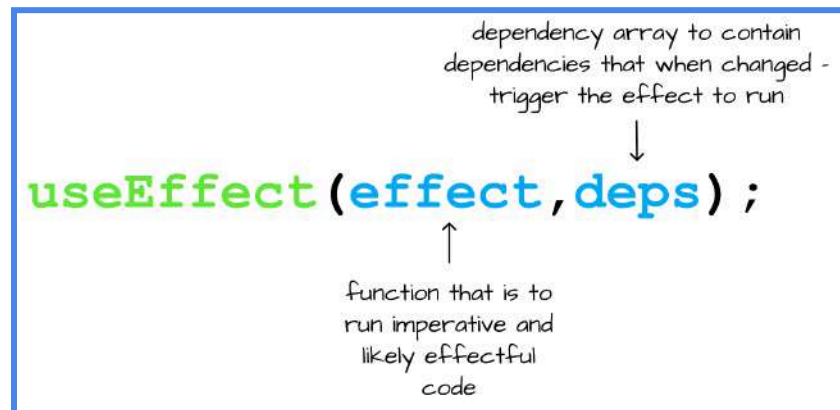


Image 2.2.1

`useEffect` adalah sebuah **Hook** yang **memungkinkan kita melakukan sesuatu** ketika sebuah **state berubah**

Pola penulisan:



Artinya: “Jalankan `someFunction` setiap kali nilai `state count` berubah.”

Jika kita menggunakan `useEffect` seperti pada contoh di atas, kita juga perlu menggunakan `useState` untuk membuat `state` terlebih dahulu. Polanya adalah: **buat state** → **state berubah** → **layar melakukan re-render** → **kemudian `someFunction` dijalankan**.



Namun, kita juga bisa menggunakan `useEffect` tanpa menggunakan `useState`! Contohnya:



```
example.tsx
useEffect(){
  someFunction,
  []
};
```

Artinya, `someFunction` hanya akan dijalankan setelah component di-render **untuk pertama kali** (*initial render*).

Ini sangat berguna ketika kalian nantinya bekerja dengan endpoint untuk fetching. **Kalian mengambil sebuah variabel dari endpoint di backend → Mengubah variabel tersebut → Mengirim perubahan tersebut kembali ke server → Layar melakukan re-render → Kemudian kalian mengambil data tersebut lagi dengan nilai yang baru.**

2.3. USEREF

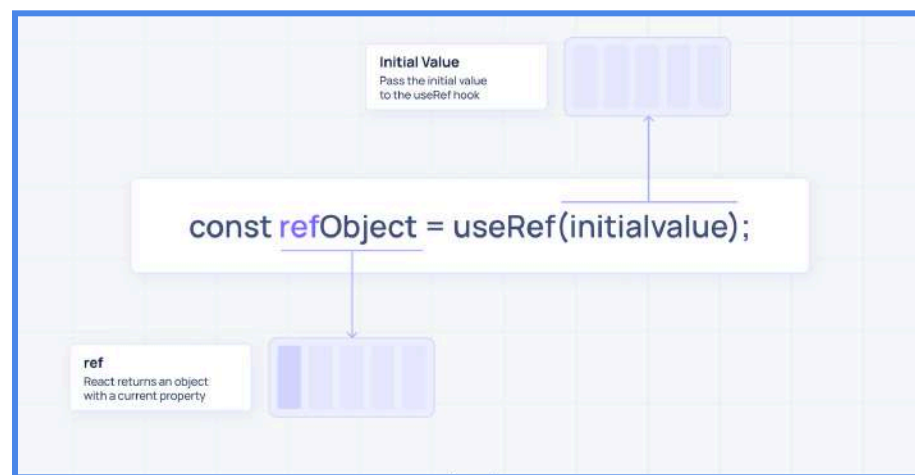


Image 2.3.1

`useRef()` digunakan ketika kalian ingin sebuah component memiliki *state* yang nilainya **dapat berubah tanpa menyebabkan component melakukan *re-render***. Jadi, berbeda dengan `useState`, mengubah nilainya tidak akan memicu *re-render* pada component.

Apakah ini berarti bahwa ketika kita mengubah nilainya, nilai tersebut sebenarnya tidak berubah? Nilainya *tetap berubah*, tetapi perubahan tersebut terjadi **di balik layar**, sehingga perubahan tersebut tidak langsung *terlihat* pada UI. Kemudian, ketika kita memicu *re-render*, nilai yang telah diperbarui akan muncul.



Lalu, apa perbedaan antara **useRef** dan **variabel biasa**? Sebuah *variable* biasa juga tidak memicu *re-render* ketika nilainya berubah, kan? Perbedaannya sederhana: ketika terjadi *re-render*, variabel biasa akan dibuat kembali dan *kembali* ke nilai yang **pertama kali kita deklarasikan**. Sementara itu, nilai yang disimpan menggunakan **useRef()** akan tetap mempertahankan nilai yang telah diubah sebelum *re-render*.

Contoh penggunaannya:

```

userref_example.tsx

import React, { useRef } from "react";
import { View, Text, Button } from "react-native";

export default function App() {
  const countRef = useRef(0);
  return (
    <View>
      <Text> Value: {countRef.current} </Text>
      <Button
        title="add"
        onPress={() => {
          countRef.current += 1;
          console.log(countRef.current);
        }}
      />
    </View>
  );
}

```



2.4. USECONTEXT

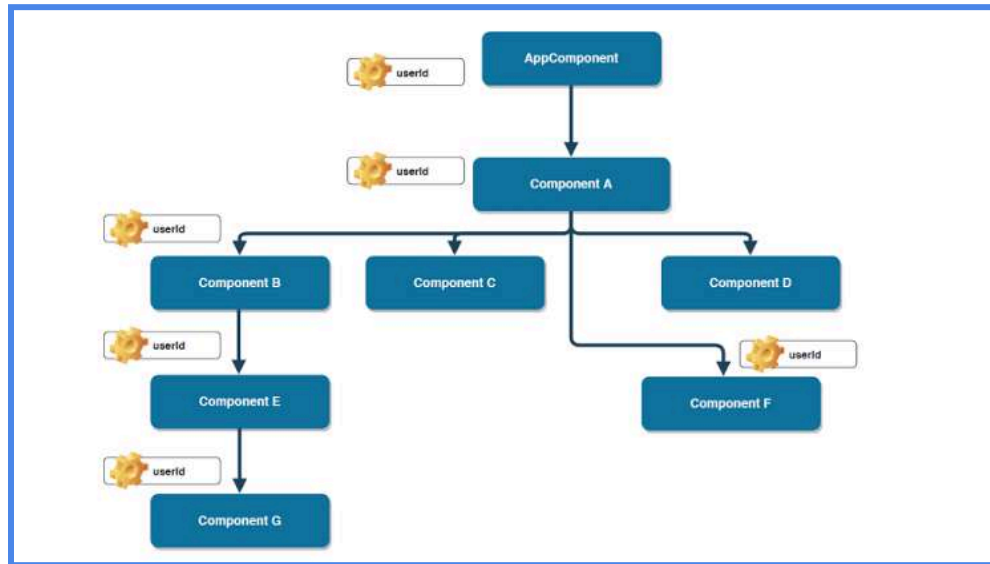
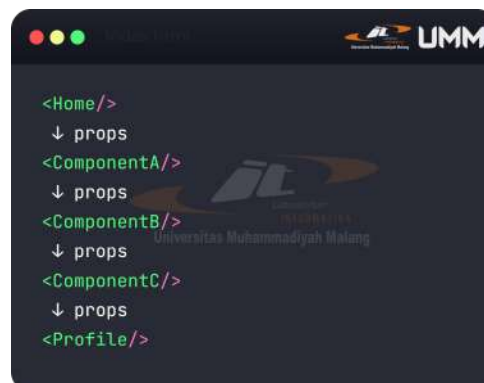


Image 2.4.1

Dalam React, kita biasanya meneruskan data dari **parent component** ke **child component** menggunakan **props**. Sebagai contoh, misalkan kita memiliki hierarki komponen berikut:



Jika **<Home/>** memiliki data yang perlu digunakan oleh **<Profile/>**, kita harus meneruskannya melalui setiap komponen:



Contoh:

```
example.tsx
<Home user={user}>
  <ComponentA user={user}>
    <ComponentB user={user}>
      <ComponentC user={user}>
        ...
      </ComponentC >
    </ComponentB>
  </ComponentA>
</Home>
```

Namun, ComponentA, ComponentB, dan ComponentC mungkin sebenarnya tidak membutuhkan data tersebut. Proses ini disebut **Prop Drilling**. **useContext** dapat digunakan untuk mengatasi masalah ini.

Apa itu Context? Context adalah *mekanisme React* untuk membagikan data kepada beberapa komponen di bawahnya tanpa harus meneruskan data tersebut secara manual melalui *props*. Ilustrasi sederhana:



Untuk membuat sebuah Context, kita menggunakan `createContext()`:

```
main.tsx

import { createContext } from "react";

const SomeContext = createContext(false);
```

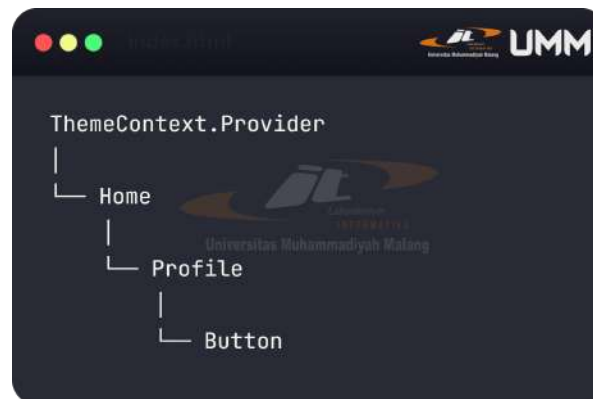
Di sini, kita membuat sebuah **Context** bernama **SomeContext**. Nilai *false* di sini adalah **nilai default** dari Context. Artinya: “Jika tidak ada Provider yang memberikan nilai, gunakan *false*.”

Setelah membuat Context, kita perlu menentukan komponen mana saja yang dapat mengaksesnya. Kita menggunakan **Provider**: `<SomeContext.Provider>`

```
main.tsx

<SomeContext.Provider value={true}>
  <Home />
</SomeContext.Provider>
```

Artinya: “Untuk semua komponen yang berada di dalam Provider ini, gunakan **true** sebagai **nilai Context**.”



Home, Profile, dan Button dapat mengakses Context.



Data yang ingin kita bagikan melalui Context diberikan menggunakan properti **value**. Kita juga dapat memberikan sebuah **object** ke dalam **value**:

```
main.tsx

<SomeContext.Provider value={{ darkMode: true, username: "Mobi" }}>
  <Home />
</SomeContext.Provider>
```

Kita bahkan dapat memberikan **fungsi (functions)**:

```
main.tsx

<ThemeContext.Provider value={{ darkMode, setDarkMode }}>
  <Home />
</ThemeContext.Provider>
```

Hal ini memungkinkan *child component* tidak hanya **membaca** data, tetapi juga **mengubahnya**.

Untuk mengambil data dari Context, gunakan: **useContext()**. Sebagai contoh:

```
main.tsx

const darkMode = useContext(SomeContext);
```

Misalkan **Provider** menyediakan:

```
main.tsx

<ThemeContext.Provider
  value={{
    darkMode,
    username,
    fontSize,
    toggleTheme,
  }}
>
  <Home />
</ThemeContext.Provider>
```



Kemudian, *value* kita adalah sebuah **object**. Sebagai contoh, nilai *defaultnya* adalah:

```
index.html
{
  darkMode: false,
  username: "Mobi",
  fontSize: 16,
  toggleTheme: () => {}
}
```

Untuk mengakses data **di dalam sebuah komponen** yang berada di dalam *Provider*:

```
main.tsx
const {
  darkMode,
  username,
  fontSize,
  toggleTheme,
} = useContext(ThemeContext);
```

Kemudian **semua value** tersebut dapat digunakan:

```
main.tsx
<Text>{username}</Text>

<Text>Font size: {fontSize}</Text>

<Button
  title="Toggle Theme"
  onPress={toggleTheme}
/>
```



CODELAB-1 (5-10 menit, kerjakan di lab)

Kita akan menggunakan salah satu contoh penggunaan yang paling umum, yaitu **Dark Mode**.
Buat sebuah file bernama **usecontext_example.tsx** dan perbaiki kode di bawah ini:

```
usecontext_example.tsx

import {_____, _____, useState} from "_____";
import {View, Text, Button} from "_____";
const ThemeContext = _____<any>(null);

export default function App() {
  const [darkMode, setDarkMode] = useState(false);
  return (
    <ThemeContext._____ value={ {darkMode, setDarkMode} }>
      <Home />
    </ThemeContext._____>
  );
}

function Home() { return <Profile />; }
function Profile() { return <UserCard />; }
function UserCard() { return <UserName />; }
function UserName() {
  const theme = _____(_____);
  const darkMode = theme.darkMode;
  const setDarkMode = theme.setDarkMode;
  return (
    <View style={ {flex: 1, backgroundColor: darkMode? "black" : "white"} }>
      <Text style={ {color: darkMode ? "white" : "black"} }>
        {darkMode ? "Dark Mode" : "Light Mode"}
      </Text>
      <Button title="Toggle Theme" onPress={() =>setDarkMode(!darkMode)} />
    </View>
  );
}
```



TIPS:

example.tsx

```
const theme = ____ (____);
const darkMode = theme.darkMode;
const setDarkMode = theme.setDarkMode;
```

Ini:

example.tsx

```
const { darkMode, setDarkMode } = ____ (____);
```

Dan ini:

Sebenarnya, keduanya **sama saja**. Pada contoh kedua (yang menggunakan kurung kurawal), kita menggunakan teknik yang disebut **object destructuring**. Artinya, kita langsung mengambil properti-properti dari object tersebut. Pada contoh pertama, kita mengambil **object-nya secara keseluruhan**, kemudian mengakses properti-propertinya melalui variabel yang telah kita buat.

2.5. USEMEMO

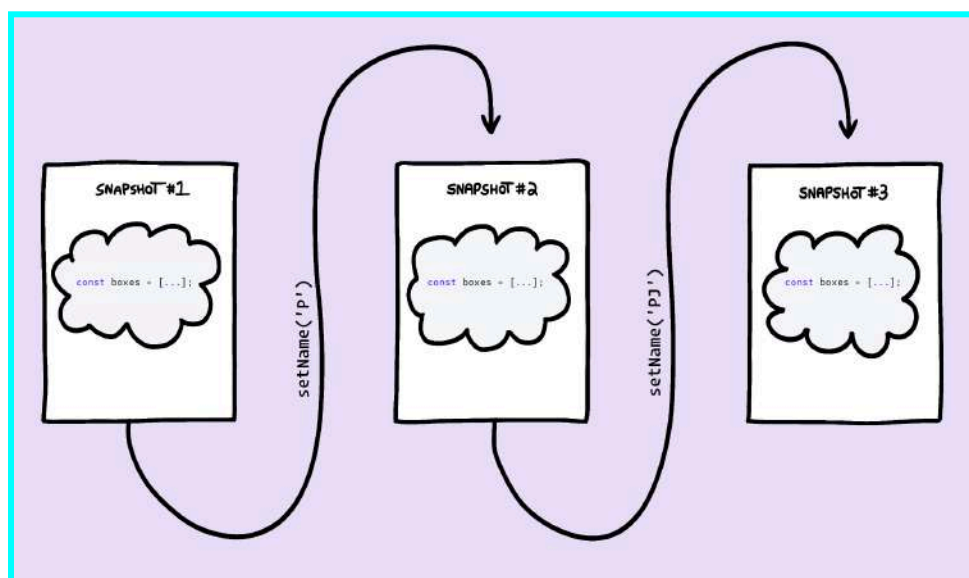


Image 2.5.1



useMemo adalah React Hook yang digunakan untuk menyimpan (memoize) hasil dari suatu proses/perhitungan sehingga React tidak perlu menghitungnya kembali setiap kali komponen melakukan re-render. Sintaks dasarnya adalah:

```
main.tsx
const result = useMemo(() => { return someCalculation(); }, [dependencies]);
```

useMemo akan menjalankan fungsi dan menyimpan hasilnya. Fungsi tersebut hanya akan dihitung kembali ketika dependensinya berubah.

Mengapa useMemo diperlukan? Perhatikan komponen berikut:

```
main.tsx
export default function App() {
  const [count, setCount] = useState(0);

  const calculate = () => {
    console.log("Calculating...");
    let result = 0;
    for (let i = 0; i < 100000000; i++) {
      result += i;
    }
    return result;
  };

  const total = calculate();

  return (
    <View>
      <Text>Count: {count}</Text>
      <Text>Total: {total}</Text>
      <Button title="Tambah" onPress={() => setCount(count + 1)} />
    </View>
  );
}
```

Ketika tombol ditekan: count berubah → komponen melakukan *re-render* → calculate() dijalankan kembali → perhitungan yang berat (*expensive calculation*) dilakukan lagi. Padahal, hanya count yang berubah.



Kita dapat menggunakan **useMemo** untuk menyimpan hasil perhitungan tersebut:

```

main.tsx

export default function App() {
  const [count, setCount] = useState(0);

  const total = useMemo(() => {
    console.log("Calculating...");
    let result = 0;
    for (let i = 0; i < 100000000; i++) {
      result += i;
    }
    return result;
  }, []);

  return (
    <View>
      <Text>Count: {count}</Text>
      <Text>Total: {total}</Text>
      <Button title="Tambah" onPress={() => setCount(count + 1)}>
    </View>
  );
}
  
```

useState, **useRef**, dan **useMemo** semuanya dapat menyimpan nilai di antara *render*, tetapi ketiganya memiliki tujuan yang berbeda:

useState → digunakan untuk menyimpan data yang ketika berubah akan memicu *re-render*, sehingga UI diperbarui.

useRef → digunakan untuk menyimpan nilai yang, ketika berubah, **tidak memicu *re-render***, sehingga cocok untuk menyimpan nilai seperti *reference* atau data sementara yang tidak perlu ditampilkan secara langsung di UI.

useMemo → digunakan untuk menyimpan hasil perhitungan agar perhitungan tersebut tidak perlu dilakukan kembali pada setiap *render*. Ketika komponen melakukan *render*, React memeriksa *dependency* dari *useMemo*. Jika suatu *dependency* berubah, perhitungan akan diulang. Jika *dependency* tidak berubah, hasil sebelumnya akan digunakan kembali.



2.6. USELOCALPARAMS()

useLocalSearchParams adalah *hook* dari Expo Router yang digunakan untuk mengambil parameter dari URL/route yang sedang aktif. Sebagai contoh, jika *route* kita adalah:



Dan kita memiliki URL: `/profile/123`. Kita dapat mengambil id yang nilainya adalah 123 di dalam kode kita. Berikut caranya:



```
const { id, name, age } = useLocalSearchParams();
return (
  <View>
    <Text>ID: {id}</Text>
    <Text>Name: {name}</Text>
    <Text>Age: {age}</Text>
  </View>
);
```

10:42 AM, Tue 12/01

UMM

University of Muhammadiyah

`/profile/123?name=Mobi&age=13`

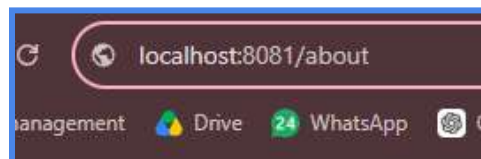
- route: profile
- dynamic parameter: id
- query parameter: name
- query parameter: age

2. PENAMAAN FILE UNTUK ROUTING

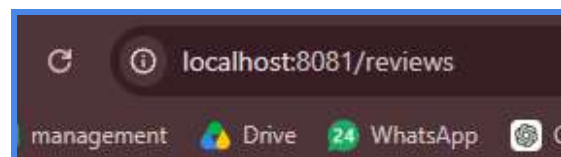


2.1. NORMAL PAGE

Halaman Normal adalah halaman biasa yang dapat diakses secara langsung dengan menggunakan nama file. Sebagai contoh, file bernama **app/about.tsx** dapat diakses melalui **/about**:



Dan file bernama **app/reviews.tsx** yang berada di dalam folder *products* dapat diakses melalui **/products/reviews**:



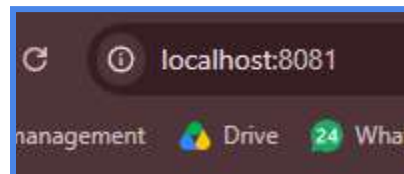
Ini adalah jenis *route* yang paling umum digunakan dalam Expo Router.



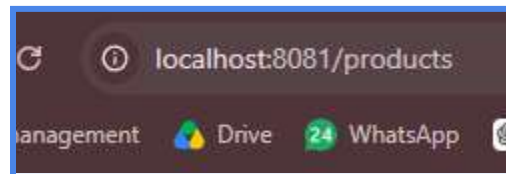
2.2. INDEX PAGE

Index Page adalah **halaman khusus yang menjadi halaman default untuk sebuah folder**. Alih-alih menggunakan *nama file*, halaman ini dapat diakses langsung melalui *route* dari folder induknya.

Sebagai contoh, **app/index.tsx** dapat diakses melalui `/`.



Sedangkan **app/products/index.tsx** dapat diakses melalui `/products`:



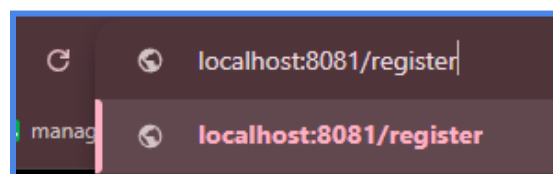
Anggap saja halaman ini sebagai “*homepage*” dari sebuah folder.

2.3. PRIVATE ROUTE

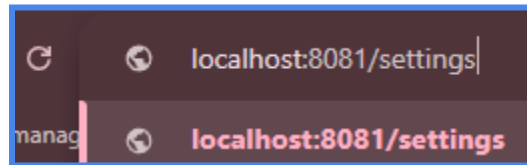
Private Route adalah **file yang namanya diawali dengan garis bawah (_)**, seperti `_layout.tsx` atau `_sitemap.tsx`. File-file ini ada di dalam project dan digunakan secara *internal* oleh Expo Router, tetapi **tidak dapat diakses secara langsung melalui URL**. File-file tersebut biasanya digunakan untuk *configuration*, *layout*, atau keperluan khusus lainnya.

2.4. ROUTE GROUP

Route Group adalah **folder yang dibungkus dengan tanda kurung**, seperti `(auth)` atau `(tabs)`. *Route group* digunakan untuk **mengorganisir halaman-halaman yang saling berkaitan tanpa memengaruhi URL route yang sebenarnya**. Sebagai contoh, file yang berada di `(auth)/register.tsx` diakses melalui `/register`, bukan `/(auth)/register`:



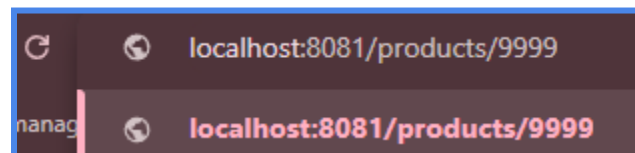
Demikian pula, `(tabs)/settings.tsx` diakses melalui `/settings`:



Nama folder yang berada di dalam tanda kurung akan diabaikan dalam URL, sehingga *route group* berguna untuk menjaga project tetap *terorganisir* tanpa mengubah struktur *routing*.

2.5. DYNAMIC ROUTE

Dynamic Route ditandai dengan **tanda kurung siku ([])**, seperti `products/[id].tsx`. Hal ini memungkinkan satu halaman menampilkan konten yang berbeda berdasarkan parameter dalam URL. Sebagai contoh, daripada mengakses `/products/[id]`, pengguna akan mengakses *route* seperti `/products/1`, `/products/2`, atau `/products/999`.



Dynamic route biasanya digunakan untuk hal-hal seperti *user profile*, *product details*, atau *blog posts*. Daripada membuat ratusan atau ribuan file terpisah, satu halaman dinamis dapat menampilkan data yang berbeda berdasarkan ID yang diberikan.

2.6. CATCH ALL ROUTE

Catch-All Route ditandai dengan `[...slug].tsx`. *Route* ini digunakan ketika kamu ingin **satu halaman menangani beberapa tingkat route**.

Sebagai contoh, daripada membuat file terpisah seperti:

- `docs/react.tsx`
- `docs/react/hooks.tsx`
- `docs/react/components.tsx`
- `docs/javascript/array.tsx`

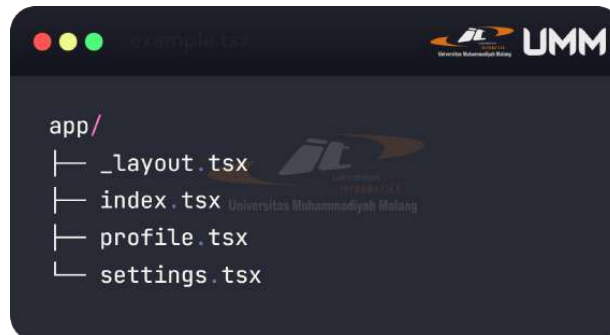
Kamu dapat membuat satu file bernama `docs/[...slug].tsx`.

Route ini kemudian dapat menangkap semua path di bawah `/docs` dan memprosesnya secara dinamis. (Kita akan mempelajarinya lebih lanjut pada *Code lab* di bawah!)

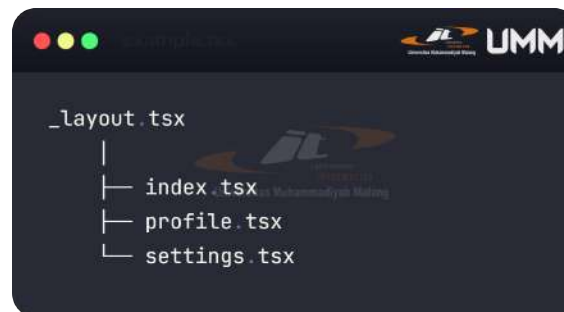


2.7. LAYOUT PAGE

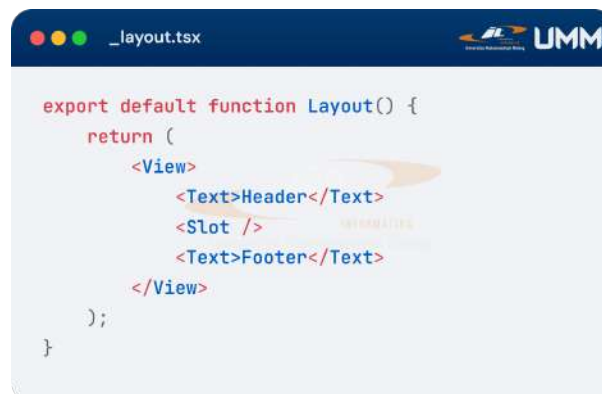
_layout.tsx adalah file yang digunakan untuk menentukan *layout* umum atau struktur yang digunakan bersama oleh halaman-halaman dalam sebuah folder. Sebagai contoh, misalkan kita memiliki:



Kemudian, **_layout.tsx** menjadi parent dari halaman-halaman tersebut:



Berikut adalah contoh sederhana **_layout.tsx** menggunakan **<Slot />**:



Artinya: "Letakkan halaman *child* di sini, di dalam **<Slot />**." Jadi, jika pengguna sedang berada di *index.tsx*, maka *index.tsx* akan di-render pada posisi **<Slot />**. Jika pengguna berpindah ke *profile.tsx*, maka *profile.tsx* akan di-render pada posisi **<Slot />**.



Jika **profile.tsx** berisi:

```

export default function Profile() {
  return (
    <View>
      <Text>Profile Page</Text>
    </View>
  );
}
  
```

Maka, yang sebenarnya di-render adalah:

```

export default function Profile() {
  return (
    <View>
      <Text>Header</Text>
      <View>
        <Text>Profile Page</Text>
      </View>
      <Text>Header</Text>
    </View>
  );
}
  
```

Hal ini berguna ketika kita ingin setiap halaman menampilkan *header* yang sama tanpa harus menambahkannya secara individual pada setiap halaman.

Sebagai contoh, misalkan kita memiliki struktur berikut di dalam **app/**:

```

app/
├─ _layout.tsx      ← Root Layout
├─ index.tsx
├─
├─ folder1/Universitas Muhammadiyah Malang
│   ├─ _layout.tsx  ← Child Layout
│   ├─ index.tsx
│   └─ profile.tsx
  
```

Child layout akan menjadi **index.tsx** dan **profile.tsx** sebagai *children*-nya. Namun, *child layout* tersebut merupakan *child* dari **root layout**.



3. NAVIGASI ANTAR PAGE

Pada bab sebelumnya, kita telah mempelajari berbagai jenis *route* yang tersedia dalam **Expo Router**, seperti *normal page*, *index page*, *route group*, *dynamic route*, dan *layout page*.

Namun, mengetahui bagaimana *route* disusun saja tidak cukup. Kita juga perlu mengetahui **bagaimana pengguna dapat berpindah dari satu halaman ke halaman lainnya** tanpa harus mengakses *route url* secara langsung.

Pada bab ini, kita akan mempelajari cara melakukan navigasi antarhalaman menggunakan **Expo Router**.

Navigasi merupakan salah satu konsep terpenting dalam pengembangan aplikasi mobile karena hampir setiap aplikasi memiliki lebih dari satu *screen*.

Sebagai contoh, pengguna dapat berpindah dari halaman **Home** ke halaman **Profile**, dari halaman **Product List** ke halaman **Product Detail**, atau dari halaman **Login** ke halaman **Register**.

Sebelum dapat melakukan navigasi antarhalaman, kita perlu memiliki beberapa halaman dalam aplikasi.

Sebagai contoh:

- `index.tsx` → `/`
- `about.tsx` → `/about`
- `profile.tsx` → `/profile`

3.1. NAVIGASI MENGGUNAKAN LINK

Cara paling mudah untuk berpindah antarhalaman adalah dengan menggunakan komponen **Link**.



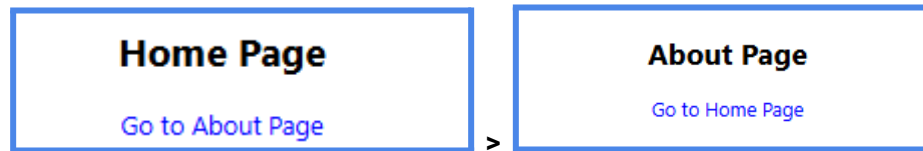
```
index.tsx
import { View, Text, StyleSheet } from "react-native";
import { Link } from "expo-router";

export default function Home() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Home Page</Text>

      <Link href="/about" style={styles.link}>
        Go to About Page
      </Link>
    </View>
  );
}
```



Output:



Ketika teks tersebut ditekan, pengguna akan diarahkan ke halaman **/about**. Anggap saja *Link* sebagai versi mobile dari tag HTML **<a>**.

3.2. NAVIGASI MENGGUNAKAN ROUTER

Meskipun **<Link>** sederhana dan praktis, terkadang kita perlu melakukan navigasi setelah tombol ditekan atau setelah menjalankan logika *async*, seperti mengambil data dari API atau memverifikasi kredensial pengguna. Untuk itu, kita menggunakan object **router** dari expo-router.

Tabs → Navigator yang menampilkan beberapa halaman dalam bentuk tab, sehingga pengguna dapat berpindah antarhalaman dengan memilih tab pada bilah navigasi. Baca selengkapnya di sini.

Drawer → Navigator yang menampilkan beberapa halaman melalui menu samping (*drawer*) yang dapat dibuka dan ditutup oleh pengguna. Baca selengkapnya di sini.

Dalam kasus ini, kita akan menggunakan **Stack**, yaitu navigator yang menampilkan halaman dalam bentuk tumpukan (*stack*), di mana halaman baru ditempatkan di atas halaman sebelumnya dan pengguna dapat kembali menggunakan tombol **Back**.

Mengapa kita mempelajari Stack terlebih dahulu?

Karena **<Stack />** adalah navigator yang paling mudah digunakan untuk memahami konsep dasar navigasi sebelum beralih ke pola yang lebih kompleks seperti *Tabs* dan *Drawer*.

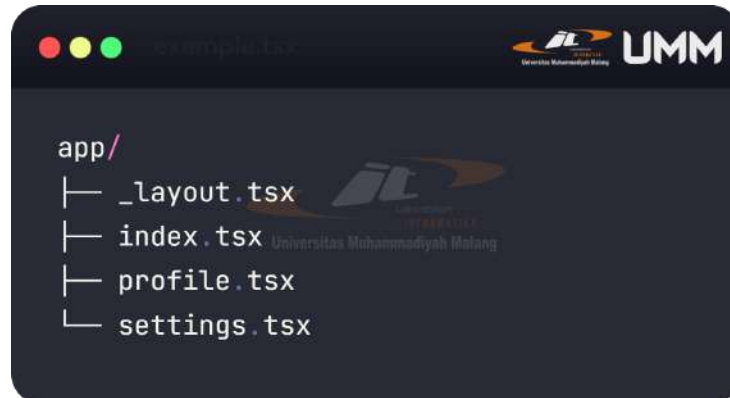
Sebenarnya, kita juga dapat mengakses dan berpindah antarhalaman menggunakan fungsi Expo Router, seperti **push()** dan **back()** (*yang akan kita bahas nanti*), tanpa secara eksplisit menggunakan *Stack*.

Namun, ada beberapa alasan mengapa kita akan menggunakan *Stack*. Kamu akan memahaminya sendiri nantinya. Jadi, terus baca materi Expo Router ini sampai selesai.

Di sini, kita akan selalu menggunakan **_layout.tsx** saat bekerja dengan Expo Router.



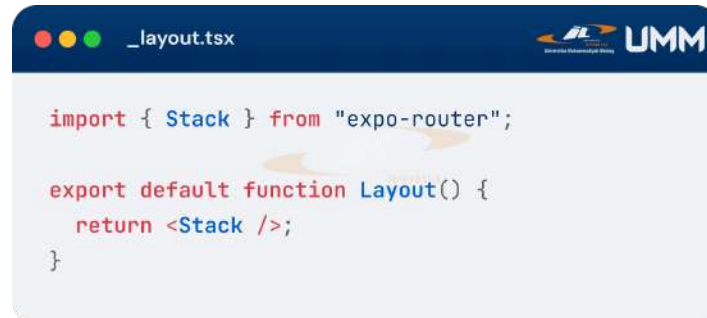
Sebagai contoh, misalkan folder **app/** berisi file-file berikut:



```

app/
├─ _layout.tsx
├─ index.tsx
├─ profile.tsx
└─ settings.tsx
  
```

Kita dapat menambahkan kode sederhana seperti berikut ke dalam **_layout.tsx** untuk menjadikan semua file tersebut sebagai bagian dari sebuah Stack:

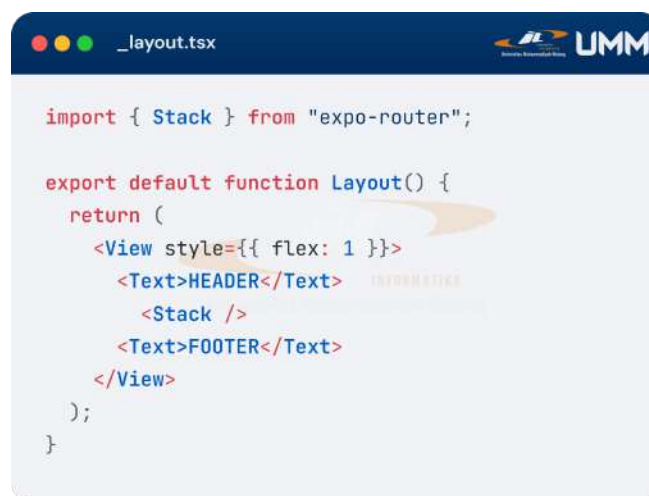


```

import { Stack } from "expo-router";

export default function Layout() {
  return <Stack />;
}
  
```

Dengan demikian, semua *children* dari **_layout.tsx** (index.tsx, profile.tsx, dan settings.tsx) dapat dikelola oleh **Stack Navigator**. Atau, jika kita juga ingin menambahkan komponen lain di sekitar *children*, kita dapat melakukan sesuatu seperti ini:



```

import { Stack } from "expo-router";

export default function Layout() {
  return (
    <View style={{ flex: 1 }}>
      <Text>HEADER</Text>
      <Stack />
      <Text>FOOTER</Text>
    </View>
  );
}
  
```



Kode di atas pada dasarnya mengatakan: *"Buat sebuah container yang memenuhi seluruh layar (flex: 1). Di dalam container tersebut, tampilkan HEADER di bagian atas, kemudian tampilkan halaman-halaman yang dikelola oleh Stack di bagian tengah, dan tampilkan FOOTER di bagian bawah."* Cara kerjanya mirip dengan `<Slot />`.

Di dalam Stack, kita juga dapat mengatur beberapa hal, seperti yang ditunjukkan pada contoh berikut:

```

import { Stack } from "expo-router";

export default function Layout() {
  return (
    <Stack>
      <Stack.Screen name="index" options={{ title: "Home" }} />
      <Stack.Screen name="profile" options={{ title: "Profile" }} />
      <Stack.Screen name="settings" options={{ title: "Settings" }} />
    </Stack>
  );
}

```

Pada dasarnya, ini sama seperti mengatakan: *"Buat sebuah Stack Navigator. Di dalamnya, kelola tiga halaman: index, profile, dan settings. Untuk halaman index, tampilkan judul Home; untuk halaman profile, tampilkan judul Profile; dan untuk halaman settings, tampilkan judul Settings."* Sekarang, tanpa menggunakan Stack, kita tidak akan bisa mengontrol navigasi seperti ini.

Contoh output:

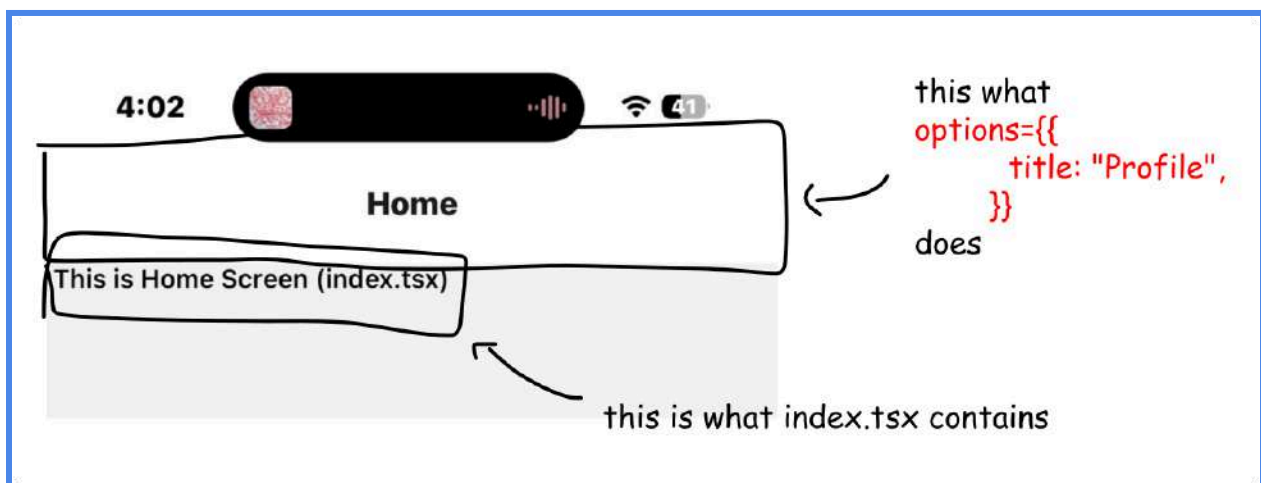


Image 3.2.1



Jadi, **<Stack.Screen />** digunakan untuk memberikan beberapa *props*, termasuk *props options* seperti yang ditunjukkan di atas. Selama kita menentukan *nama file* pada *props name*.

Baiklah, mari kita lanjutkan dengan membahas fungsi-fungsi yang disediakan oleh Expo Router untuk membantu kita berpindah antarhalaman.

A. PUSH() & BACK()

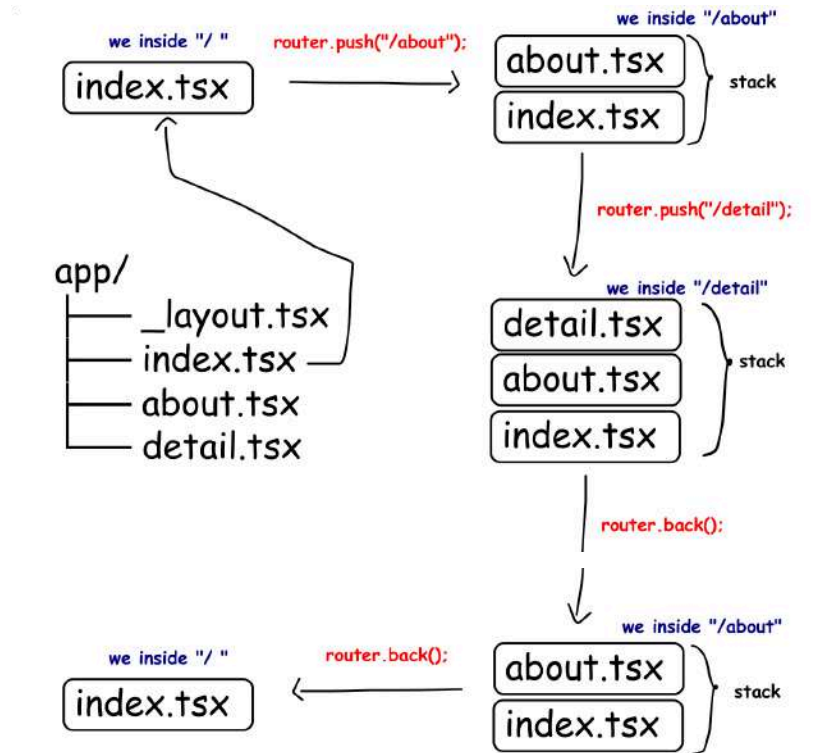


Image 3.2.A.1

Metode **router.push()** menambahkan screen baru ke bagian paling atas *navigation stack*. Karena *screen* sebelumnya tetap berada di bawahnya, pengguna dapat dengan mudah kembali menggunakan metode **router.back()**. Metode ini biasanya digunakan ketika membuka halaman detail dari sebuah list (misalnya berpindah dari `/products` ke `/products/12`), melanjutkan ke langkah berikutnya dalam *multi-step form*, atau mengunjungi halaman profil pengguna ketika pengguna perlu kembali ke halaman sebelumnya.



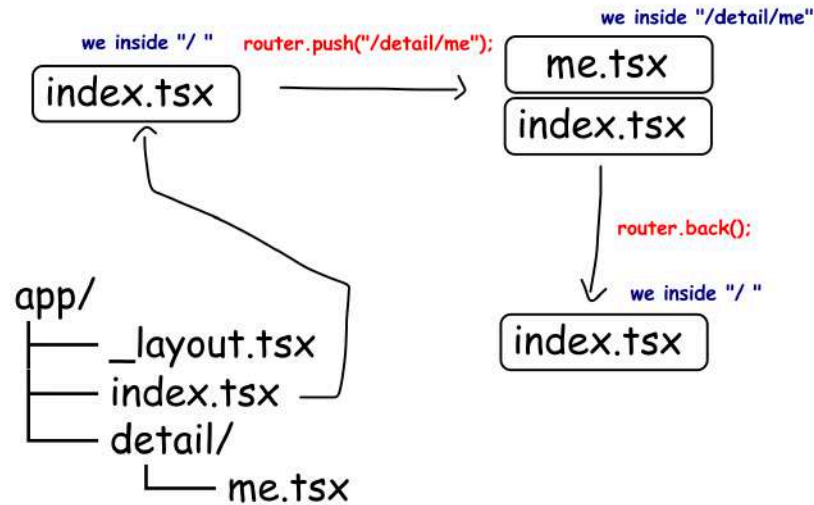


Image 3.2.A.2

B. REPLACE()

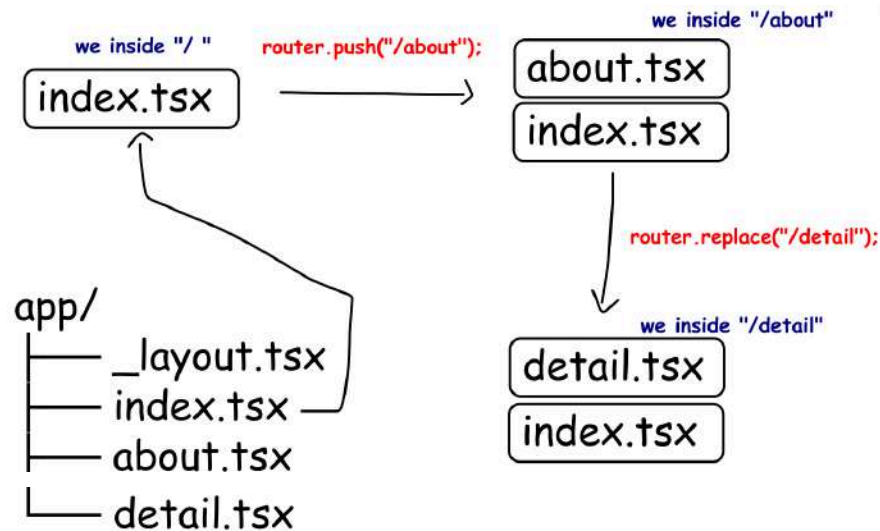


Image 3.2.B.1

Sebaliknya, **router.replace()** menggantikan screen saat ini di dalam stack dengan screen baru, sehingga halaman sebelumnya langsung dihapus dari riwayat navigasi.

Hal ini penting untuk digunakan setelah pengguna melakukan *login* atau menyelesaikan *onboarding*, karena kita tidak ingin pengguna menekan tombol *back* dan secara tidak sengaja kembali ke halaman *login* atau *splash screen*. Metode ini juga cocok digunakan pada halaman penyelesaian pesanan setelah proses *checkout*.

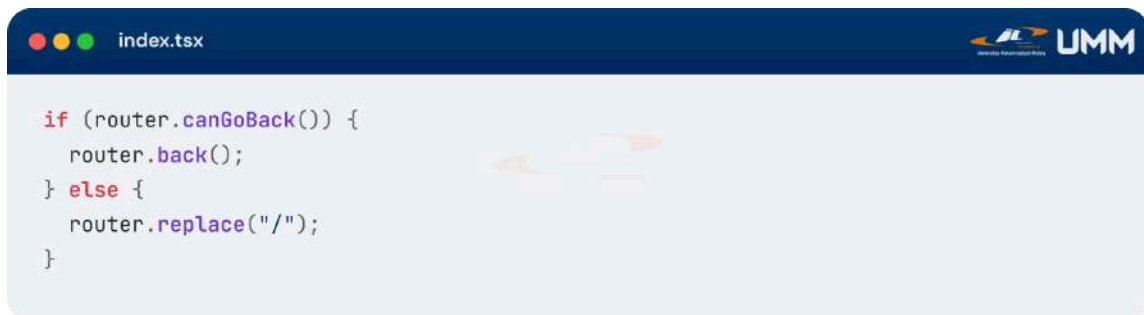


C. CANGOBACK()

Sebelum mencoba kembali ke halaman sebelumnya, **router.canGoBack()** memeriksa apakah memang ada *screen* di dalam riwayat *stack* yang dapat dikembalikan. Fungsi ini akan menghasilkan nilai **true** atau **false**.

Hal ini sangat berguna ketika menangani *deep link* atau *push notification*, di mana pengguna mungkin membuka *sub-screen* secara langsung tanpa memiliki riwayat halaman sebelumnya untuk kembali.

Dengan pemeriksaan ini, kita dapat memanggil **router.back()** dengan aman atau menggunakan **router.replace("/")** sebagai *fallback*, sehingga pengguna tidak pernah terjebak di halaman yang tidak memiliki tujuan berikutnya.



```

if (router.canGoBack()) {
  router.back();
} else {
  router.replace("/");
}

```

D. DISMISS()

Berbeda dengan *router.back()*, **dismiss()** digunakan untuk menutup *screen* atau *modal* yang sedang aktif dan kembali ke navigator sebelumnya. Oke, mari kita lihat ilustrasi berikut sebagai contoh perbedaannya:

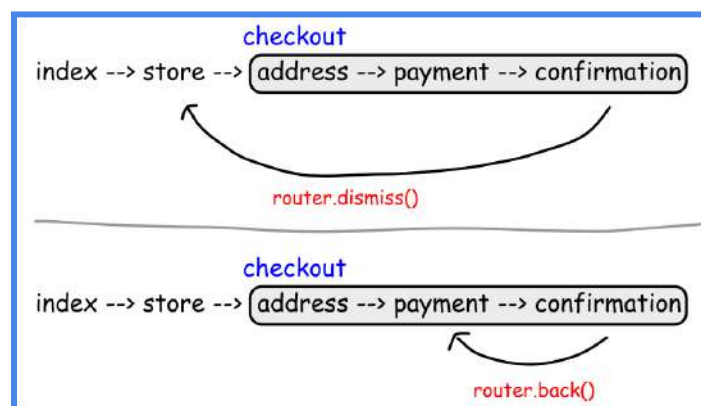


Image 3.2.D.1



Untuk melakukan hal tersebut, kita perlu mengatur file seperti berikut:

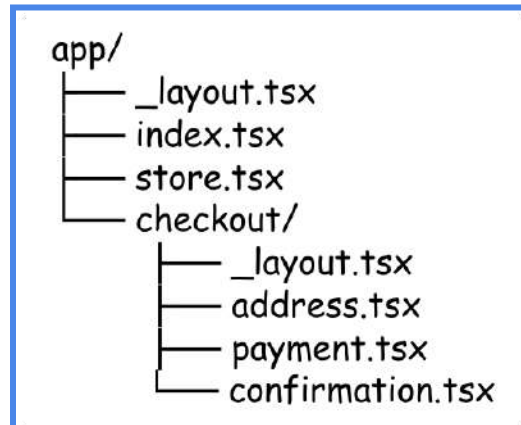


Image 3.2.D.2

Kita dapat melihat bahwa terdapat 2 file **_layout.tsx** di sini. Satu berada di dalam **app/**, sedangkan yang lainnya berada di dalam **app/checkout/**.

Isi Layout 1 (di dalam app/):



```

export default function RootLayout() {
  return (
    <Stack>
      <Stack.Screen name="index" options={{title: "Home"}} />
      <Stack.Screen
        name="checkout"
        options={{presentation: "modal", headerShown: false}}
      />
    </Stack>
  );
}

```

Ini berarti kita memperlakukan folder *checkout* sebagai **Stack baru**. Stack tersebut juga memiliki Stack-nya sendiri. Hal ini disebut **nested Stack**.



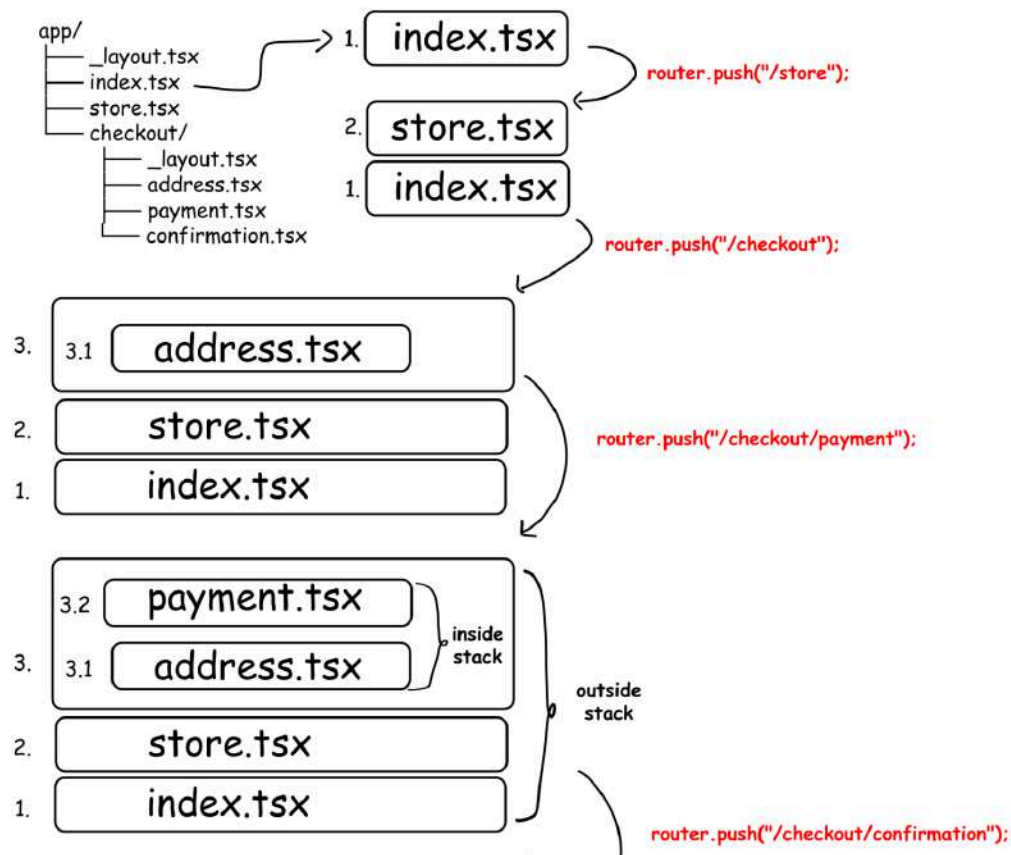
Isi checkout/_layout.tsx:

```

import { Stack } from "expo-router";

export default function CheckoutLayout() {
  return (
    <Stack>
      <Stack.Screen name="address" options={{title: "Address"}} />
      <Stack.Screen name="payment" options={{title: "Payment"}} />
      <Stack.Screen name="confirmation" options={{title: "Confirmation"}} />
    </Stack>
  );
}

```



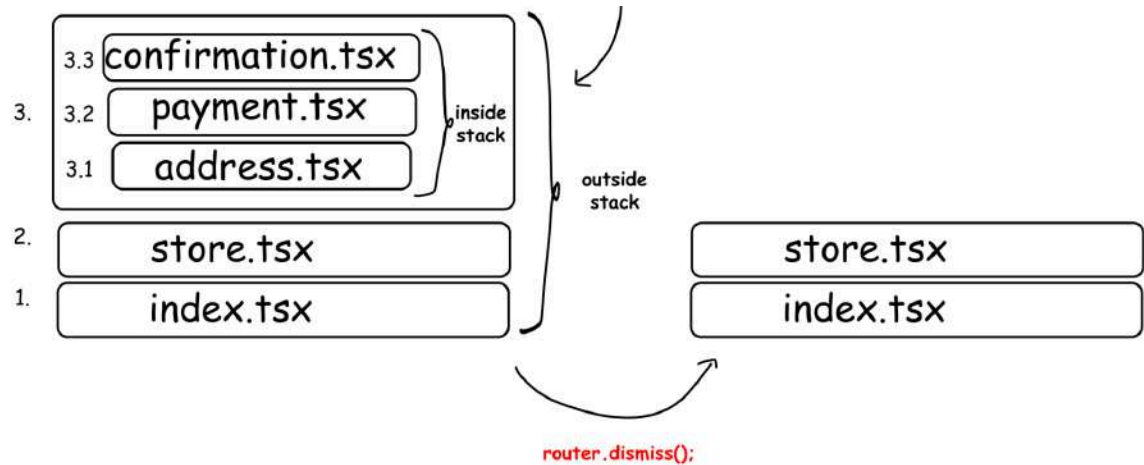
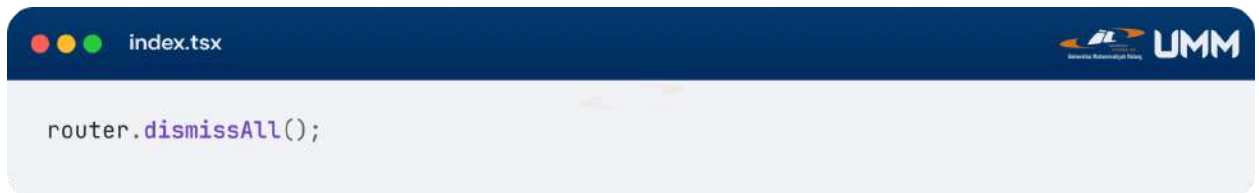


Image 3.2.D.3

Tanpa `presentation: "modal"`, hal ini tidak akan berfungsi karena akan dianggap sebagai satu stack.

E. DISMISSALL()

Ketika terdapat beberapa *nested modal* yang terbuka satu di atas yang lain dan kamu ingin menutup semuanya dengan satu kali tap, `router.dismissAll()` akan menghapus semua *modal stack* yang sedang aktif sekaligus dan mengembalikan pengguna langsung ke *screen* utama di belakangnya.



Atau jika kamu ingin menutup beberapa *screen* sekaligus hingga mencapai *route* tertentu: `router.dismissTo("/home");`



G. SETPARAMS()

`route.setParams()` digunakan untuk mengubah atau menambahkan parameter pada *route* yang sedang aktif dalam React Navigation / Expo Router. Sebelumnya, pada `route.push()`, kita memiliki:

```
main.tsx
router.push({
  pathname: "/profile",
  params: { id: "123", name: "Mobi" },
});
```

Sebagai contoh, kamu sedang berada di halaman: `/profile?id=123&name=Mobi`. Kemudian, kamu ingin mengubah `name` menjadi `"Luffy"` tanpa berpindah ke halaman baru:

```
main.tsx
route.setParams({ name: "Luffy" });
```

Jadi, `setParams()` tidak digunakan untuk berpindah antarhalaman. Fungsi ini hanya digunakan untuk mengubah parameter pada *route* yang sedang aktif.



4. PRAKTIK

Practice: Navigation & Catch-All Route — Expo Router

Tujuan: Membuat aplikasi dokumentasi sederhana dengan navigasi antarhalaman, *dynamic routes*, dan *catch-all routes*.

Silakan buat project baru dan buat struktur folder berikut:



Kemudian, buka **styles/page.ts** dan tambahkan *style* berikut: [Tautan](#).

Buka **styles/button.ts** dan tambahkan *style* berikut: [Tautan](#).

Selanjutnya, buka **styles/index.ts** dan tambahkan kode sederhana berikut:



Setelah itu, buka **components/pageButton.tsx** dan tambahkan kode berikut: [Tautan](#).



CODELAB-2 (10 menit, kerjakan di lab)

Ikuti semua langkah praktik di atas, lalu buka file **app/_layout.tsx**. Lengkapi kode berikut:

```

_layout.tsx

import { _____ } from "_____";

export default function RootLayout() {
  return (
    <Stack>
      <_____.Screen _____="_____" options={ { _____: "Home" } }/>
      <Stack._____ name="docs" _____={ {title: "Documentation" } }/>
    </_____>
  );
}
  
```

Selanjutnya, buka **app/index.tsx**. Lengkapi kode berikut:

```

index.tsx

import { View, Text } from "react-native";
import { useRouter } from "expo-router";
import PageButton from "@components/PageButton";
import { pageStyles } from "@styles";

export default function HomePage() {
  const router = _____();

  return (
    <View style={pageStyles.container}>
      <Text style={pageStyles.title}>Page Explorer</Text>

      <Text style={pageStyles.description}>Latihan Expo Router dan Styling</Text>

      <PageButton title="Open Documentation" onPress={ () => router.push("/docs") }/>
    </View>
  );
}
  
```



```

    { /* INLINE STYLE */
    <Text
      style={{
        textAlign: "center",
        color: "#999999",
        marginTop: 10,
      }}
    >
      Learn Expo Router
    </Text>
  </View>
);
}

```

Setelah selesai, lanjutkan praktik di bawah ini hingga selesai.

Baiklah, mari kita lanjutkan. Buka **app/docs/index.tsx** dan tambahkan kode dari tautan berikut: [Tautan](#).

Terakhir, buka **app/docs/[...slug].tsx** dan tambahkan kode dari tautan berikut: [Tautan](#).

Seperti yang dapat kamu lihat, kita mengambil **slug** menggunakan **useLocalSearchParams**. *Slug* yang kita terima akan berupa *array* yang kemudian digabungkan menjadi satu *path* menggunakan */* sebagai pemisah. Konten yang ditampilkan kemudian akan disesuaikan dengan *path* saat ini.

Kita juga menggunakan **external styling** dan **external components**. Perhatikan bahwa kita mengimpornya menggunakan **@**. Ini berarti simbol **@** merupakan *alias* untuk **root directory** dari project.

File **index.ts** berfungsi sebagai *entry point* untuk sebuah folder. Artinya, jika kita ingin mengimpor sesuatu dari folder tersebut, kita cukup mengimpornya melalui **@/styles** daripada mengimpor setiap file secara individual.



TUGAS PEKAN DEMO

1. DESKRIPSI

1. Buat kelompok & pilih tema:

- Buat kelompok yang terdiri dari 3 anggota dan pilih tema project kalian sendiri melalui tautan [di sini](#). *(Tema project harus unik di seluruh kelas. Jika terdapat dua kelompok yang memilih tema yang sama, kelompok yang mendaftarkannya terlebih dahulu akan mendapatkan tema tersebut.)*
- Untuk tugas pada modul-modul berikutnya, kalian akan bekerja bersama anggota kelompok di rumah untuk mengembangkan project berdasarkan tema tersebut.
- Untuk melihat kontribusi setiap anggota kelompok, pastikan kalian bekerja bersama menggunakan lingkungan GitHub agar kami dapat melihat bahwa beban kerja dan tugas telah dibagi secara adil kepada semua anggota (tidak ada satu orang yang mengerjakan semuanya sendirian).
- Tugas ini akan berlanjut sepanjang Modul 1–6. Kalian akan terus mengembangkan project yang sama dari modul sebelumnya. Pastikan semua modul sebelumnya dapat berjalan dengan baik karena semuanya juga akan diperiksa pada akhir perkuliahan.

2. Buat aplikasi kalian bersama:

- Tugas kalian adalah menerapkan semua yang telah dipelajari dalam modul ini ke dalam satu aplikasi yang dapat berjalan (**perhatikan kriteria penilaian dengan saksama**).
- Pastikan aplikasi kalian mengikuti tema yang telah dipilih.
- Kalian bebas menerapkan materi yang telah dipelajari di bagian mana pun dalam kode. Pastikan penerapannya masuk akal dan dilakukan dengan benar.
- Kalian tidak perlu melakukan *build* aplikasi. Namun, jika ingin melakukannya, tentu saja diperbolehkan. Kalian juga dapat menggunakan Expo Go dengan menjalankan: `npx expo start --go`.
- Saya menyediakan contoh aplikasi dasar berikut: [Tautan](#) (akan di up setelah masuk pekan materi modul 2).

3. Di pekan demo, kalian perlu:

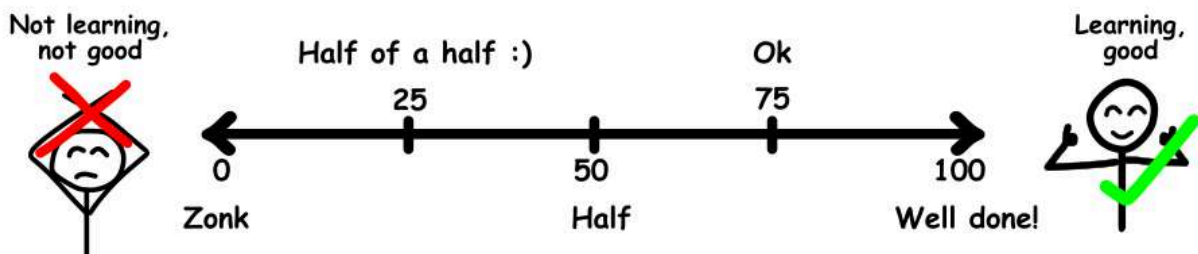
- Menunjukkan hasil pekerjaan kalian kepada asisten praktikum (**secara individu**).
- Menjawab pertanyaan yang diberikan oleh asisten praktikum (**secara individu**).
- Melakukan modifikasi kode secara langsung sesuai permintaan asisten praktikum (**secara individu**).
- Praktikan tidak diperkenankan membuka modul atau menggunakan fitur AI pada code editor (inline maupun chatbot agent).

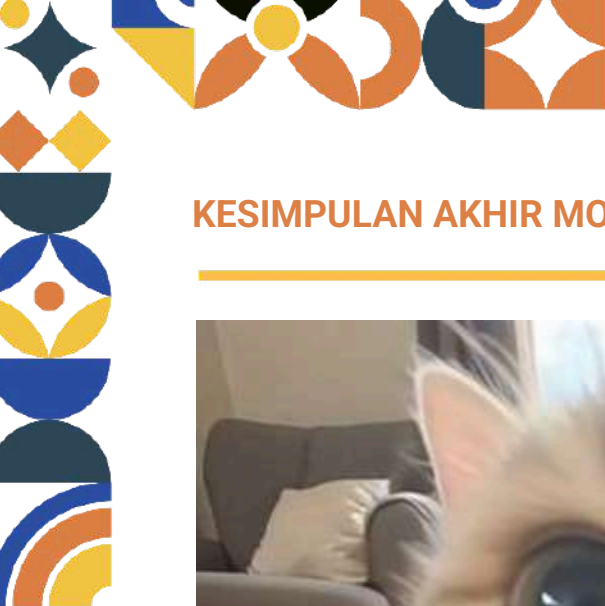


2. KRITERIA PENILAIAN

Assessment Criteria	Assessment Weight
Codelab 1-2	40%
Tugas Pekan Demo	60%
Menerapkan hook useState dan useEffect().	10%
Menerapkan push(), back(), dan 1 fungsi expo-router lainnya yang dijelaskan dalam modul.	10%
Menerapkan _layout.ts dan <Stack.Screen>.	10%
Menjawab Pertanyaan.	15%
Menyelesaikan Modifikasi Kode.	15%
TOTAL	100%

3. SKALA PENILAIAN





KESIMPULAN AKHIR MODUL

